



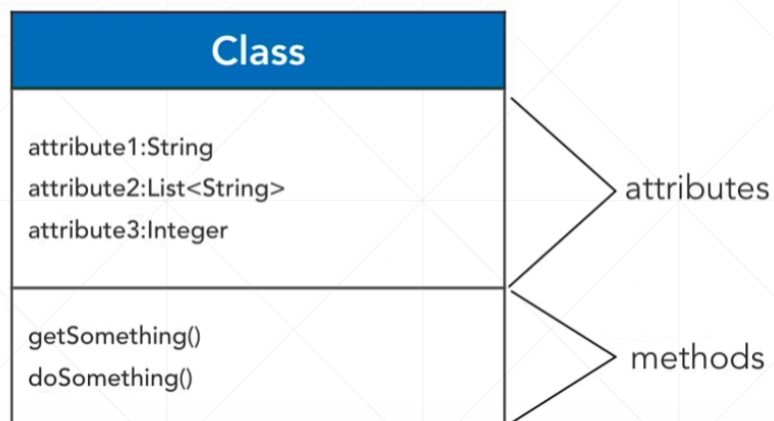
Java软件系统代码的组织方式

北京理工大学计算机学院
金旭亮

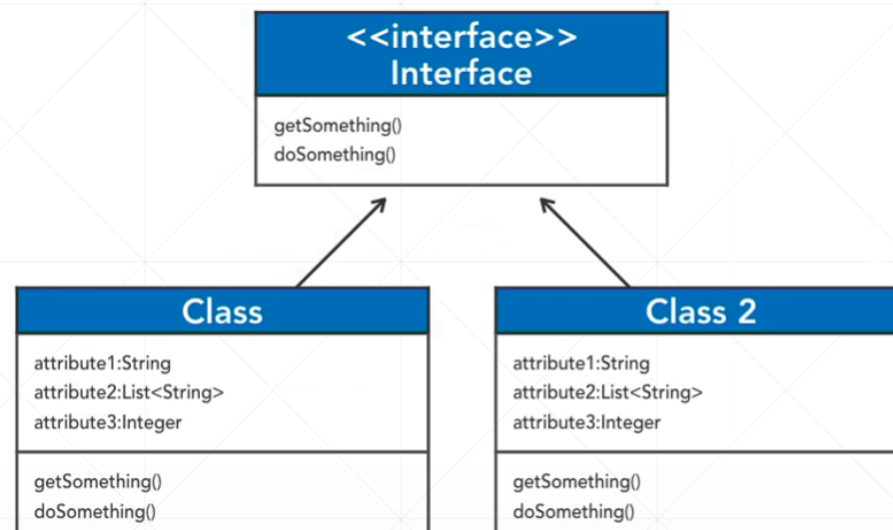


从1995年Java正式发布开始，Java走过了近三十年的漫长发展历程，在这个历程中，Java软件系统的构建，始终遵循面向对象的基本原则，采用典型的面向对象软件开发模式.....

以类作为软件系统最底层的基础构件

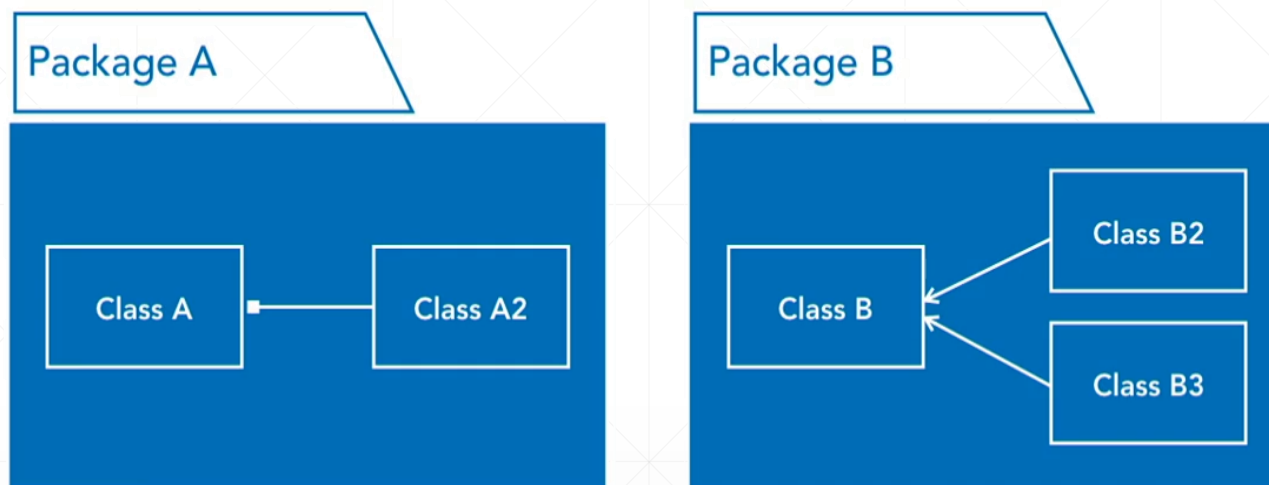
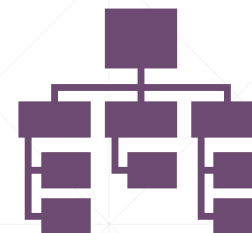


类，天生就有“模块化”的特征



通过引入“接口”，我们让类可以方便地替换，从而为“基于代码重用”的软件构建方式开辟了道路

通过包来组织和管理类



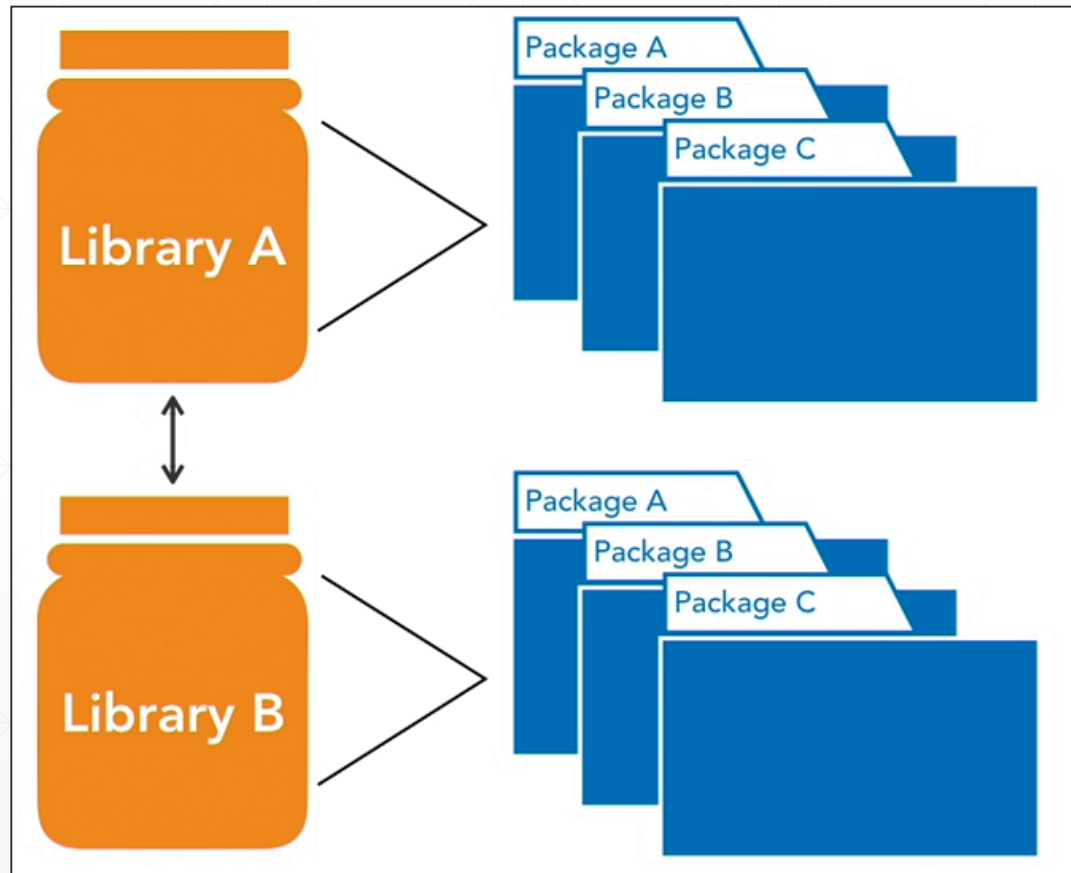
当类数量比较多以后，将其按照特定的标准（比如类所实现职责之间的关联性）进行分类，为每个类别定义一个“包（package）”，从而实现了类的“分类化”管理。

JAR包是Java组件和应用程序的“物理”载体

Java项目主要使用JAR包作为基本（物理）构件。

JAR包内可以包容多个package，每个package包容多个类，package中类的依赖关系，决定了JAR包之间的“依赖”关系。

JAR包引入，在“二进制”层次实现了代码重用，使得“搭积木”式的组件化开发模式成为可能。

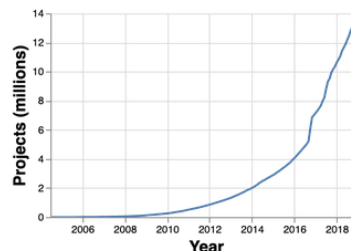


使用构建系统管理可重用的Java组件

maven



Indexed Artifacts (24.8M)



Popular Categories

- Aspect Oriented
- Actor Frameworks
- Application Metrics
- Build Tools
- Bytecode Libraries
- Command Line Parsers
- Cache Implementations
- Cloud Computing

What's New in Maven



Quarkus JAX RS Client Reactive Deployment

6 usages

[io.quarkus](#) » [quarkus-jaxrs-client-reactive-deployment](#) » 2.5.4.Final

Apache

Quarkus JAX RS Client Reactive Deployment

Last Release on Dec 15, 2021



Quarkus Elasticsearch REST Client Common Deployment

5 usages

[io.quarkus](#) » [quarkus-elasticsearch-rest-client-common-deployment](#) » 2.5.4.Final

Apache

Quarkus Elasticsearch REST Client Common Deployment

Last Release on Dec 15, 2021

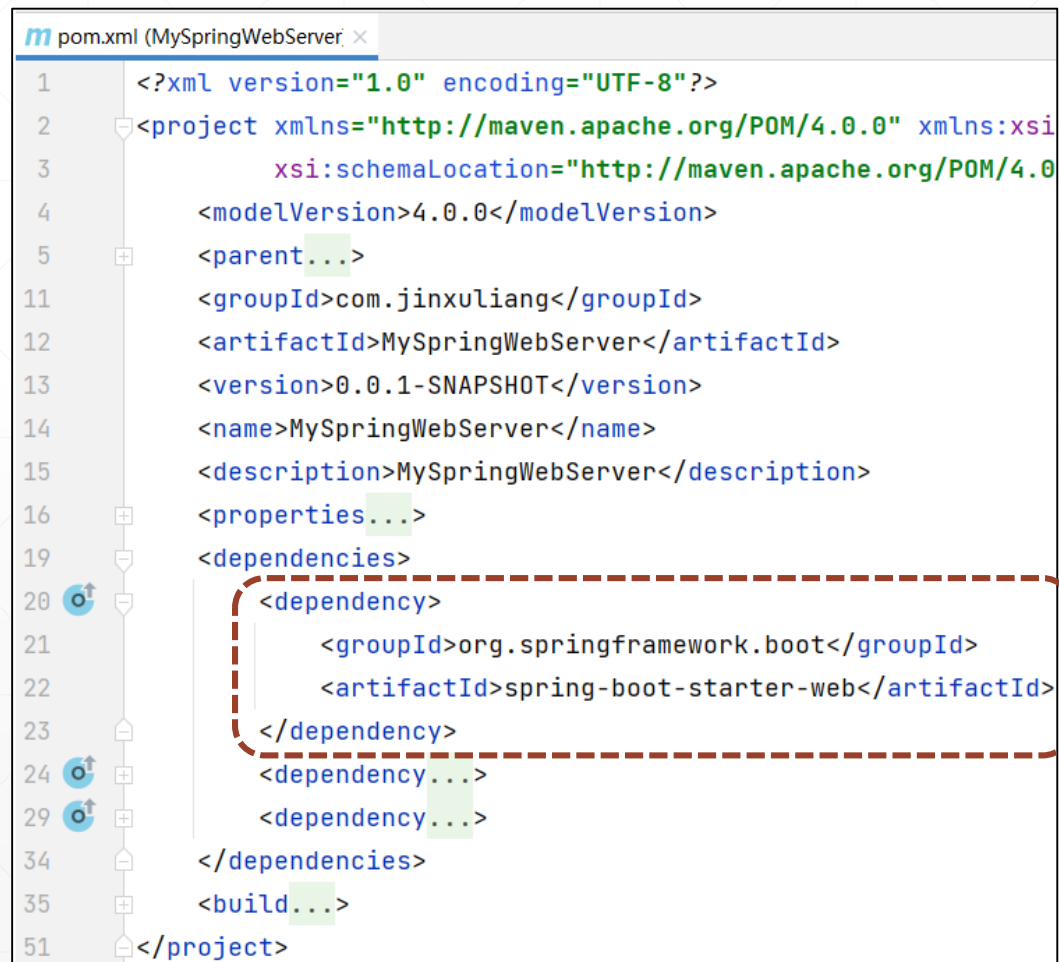
MvnRepository网站上，集中了全世界开发者开发的，总数上千万的可重用JAR包。

构建工具的依赖关系管理功能

使用Maven/Gradle构建工具所解决的一个问题是实现编译时依赖关系管理。

JAR包之间的依赖关系，Maven项目放在POM（Project Object Model，项目对象模型）文件中定义。

如果使用Gradle构建，则对应的配置文件名为build.gradle。



The screenshot shows a Maven POM file for a project named 'MySpringWebServer'. The file is an XML document with the following structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://maven.apache.org/POM/4.0.0"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/POM/4.0.0/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>...</parent>
    <groupId>com.jinxuliang</groupId>
    <artifactId>MySpringWebServer</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>MySpringWebServer</name>
    <description>MySpringWebServer</description>
    <properties>...</properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>...</dependency>
        <dependency>...</dependency>
    </dependencies>
    <build>...</build>
</project>
```

The dependency on 'org.springframework.boot:spring-boot-starter-web' is highlighted with a red dashed box.

多了解一点：OSGi

OSGi是一个致力于将Java应用模块化的技术框架：



OSGi要求将导入的包在JAR中列为元数据，称之为**捆绑包 (bundle)**。此外，还必须显式定义导出哪些包 (package)，即对其他捆绑包可见的包 (package)。在系统启动时，OSGi框架会对这些包 (package) 之间的依赖关系进行检查。

问：

有了Maven和OSGi，为什么还要引入JPMS？



答：



Maven解决的是**编译时**的组件依赖与版本问题，而且并没有彻底解决。



OSGi倒是解决得比较彻底，但却由于其复杂性，没有真正地使用开来，相反，Maven借助于MavenRepository这样的中心组件仓库，成为事实上的标准。



JPMS则意图同时在**编译时**和**运行时**两块同时发力，解决Java早期版本中存在的各种问题。

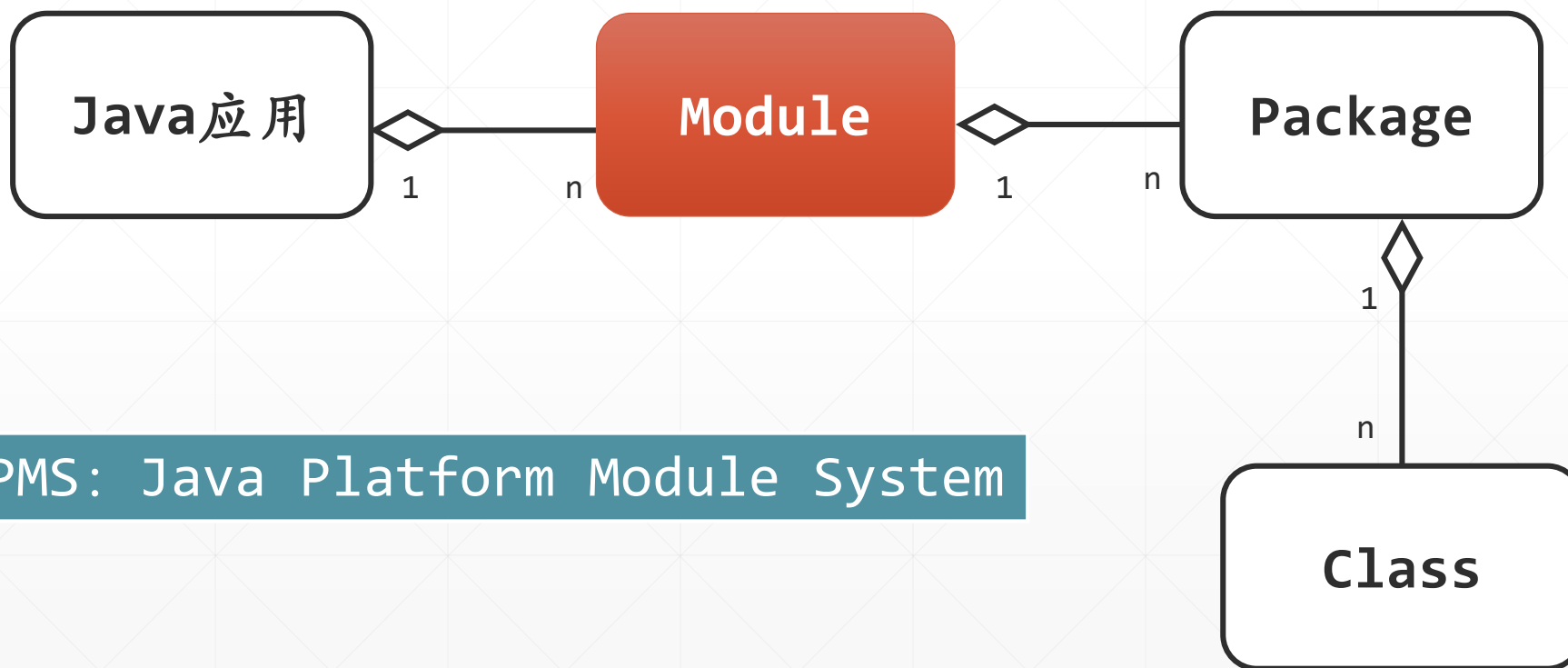
JDK9，引入模块作为应用的基本构建单元



“模块 (Module)” 是Java平台模块系统 (JPMS) 的基石。像JAR一样，模块也是类型和资源的容器，但相比JAR，它们还具有额外的特性：

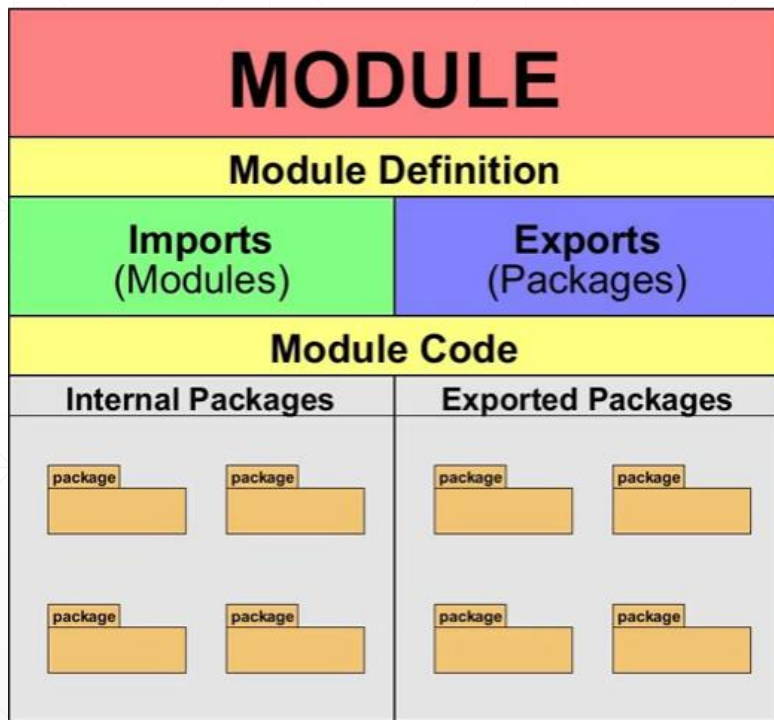
1. 必须有一个名称（全局唯一）
2. 必须显式声明对其他模块的依赖；
3. 定义了一个由导出包（exported package）构成的 “应用程序编程接口（API）”。

模块化的Java应用构建层次图



JPMS: Java Platform Module System

Java模块的内部结构图



Java模块具有很强的封装性，即使是公有类型，如果它所在的包没有导出的话，外部是不能访问它们的。

比对Module和JAR包

JAR	MODULE
包容多个package，每个package包容多个类	同JAR
没有办法确定JAR之间的依赖关系	模块配有一个配置文件，其中定义了模块之间的依赖关系。
只有在运行时，类之间的依赖关系才会被正确确定，如果这种依赖关系无法被满足，将抛出运行时异常。	在编译时就能进行检查，模块间如果存在无法满足的依赖关系，会被提前发现并提醒开发者更正。同样的检查，这种检查也会在运行时再执行一遍，更为安全。

小结



本讲介绍了Java软件系统的代码组织与管理方式，可以看到，模块化是一个重大变化，它对Java软件系统的构建，以及后继Java平台技术的演进，有着深远的影响。

可以这么说，掌握Java模块化技术，已经成为当前Java程序员必备的职业开发技能。

下一讲



模块化后的JDK